Python I. beadandó feladat

ELTE IK PNYFT – 2026 C feladat

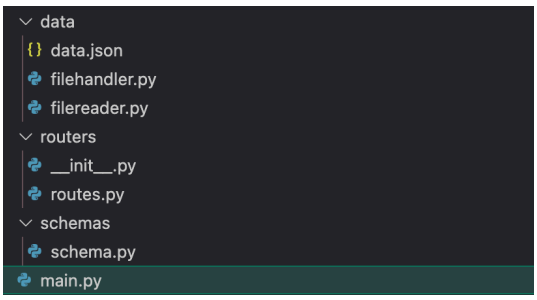
REST API készítése FastAPI alapokon – Kurzusfelvételi rendszer

A beadandó feladatban elkészítendő API egy egyszerűsített, Neptunhoz hasonló tanulmányi rendszer háttérszolgáltatása. Lehetővé teszi hallgatók felvételét a rendszerbe, egy félév (leckekönyv-bejegyzés) hozzárendelését egy adott hallgatóhoz, valamint tárgyak felvételét a hallgató félévébe az API végpontok segítségével.

A felvett tárgyak adatai módosíthatók (például így lehet beírni az érdemjegyet), a tárgyak le is adhatók. Egy félévben legfeljebb 30 kreditnyi tárgy vehető fel, és egy tárgy csak egyszer szerepelhet a félévben. Le lehet kérdezni egy hallgató kreditstatisztikáját: a felvett és a teljesített kreditek számát, valamint a kreditsúlyozott átlagot. Az API ezeken kívül lehetővé teszi egy adott hallgató adatainak lekérdezését, az összes hallgató megjelenítését és egy hallgató félévében felvett tárgyak listázását. Ezekkel a funkciókkal elkészíthető egy tanulmányi rendszer webes felülete is. A felhasználói felületet nem kell elkészíteni, de ha az API-hoz készül ilyen felület, az extra pontot érhet.

Az API a végpontok által kezelt adatokat a data.json nevű fájlban tárolja. A fájl tartalma az erre a célra kialakított végpontokkal menthető és visszatölthető. A feladat tehát egy FastAPI implementáció kiegészítése. A kód egy része már meg van írva a mellékelt könyvtárakban, az előre elkészített fájlokban pedig instrukciók segítik az adott modul funkcióinak megírását és véglegesítését.

A mellékelt fájlok tartalma



- A data.json az adatok tárolására szolgál JSON formátumban. Ebben elhelyeztünk néhány tesztadatot (3 hallgató, 3 félév), amelyekkel az alkalmazás kipróbálható. A fájl két listát tartalmaz: `Students` és `Semesters`.
- A filehandler.py és a filereader.py a JSON adatok fájlba írására, illetve fájlból történő beolvasására használható. A függvények fejlécei és a használatukat bemutató példák már szerepelnek a fájlokban, a függvények törzsét kell megírni.

- A routes.py tartalmazza az alkalmazás végpontjait. Ezeket a függvényeket kell kiegészíteni a megfelelő funkciókkal és paraméterekkel.
- A schema.py a JSON adatok kliens és szerver közötti mozgatására, valamint kezelésére alkalmas osztályokat tartalmazza. Ezekben a mezők már fel vannak sorolva, de a validáció még hiányzik. Az osztályokat a fájlban leírtak alapján ki kell egészíteni, ügyelve az adatok helyességére.
- A main.py tartalmazza a FastAPI main modulját, amelyet a uvicorn Python csomag segítségével kell futtatni az alkalmazás elindításához.

Az adatok szerkezete

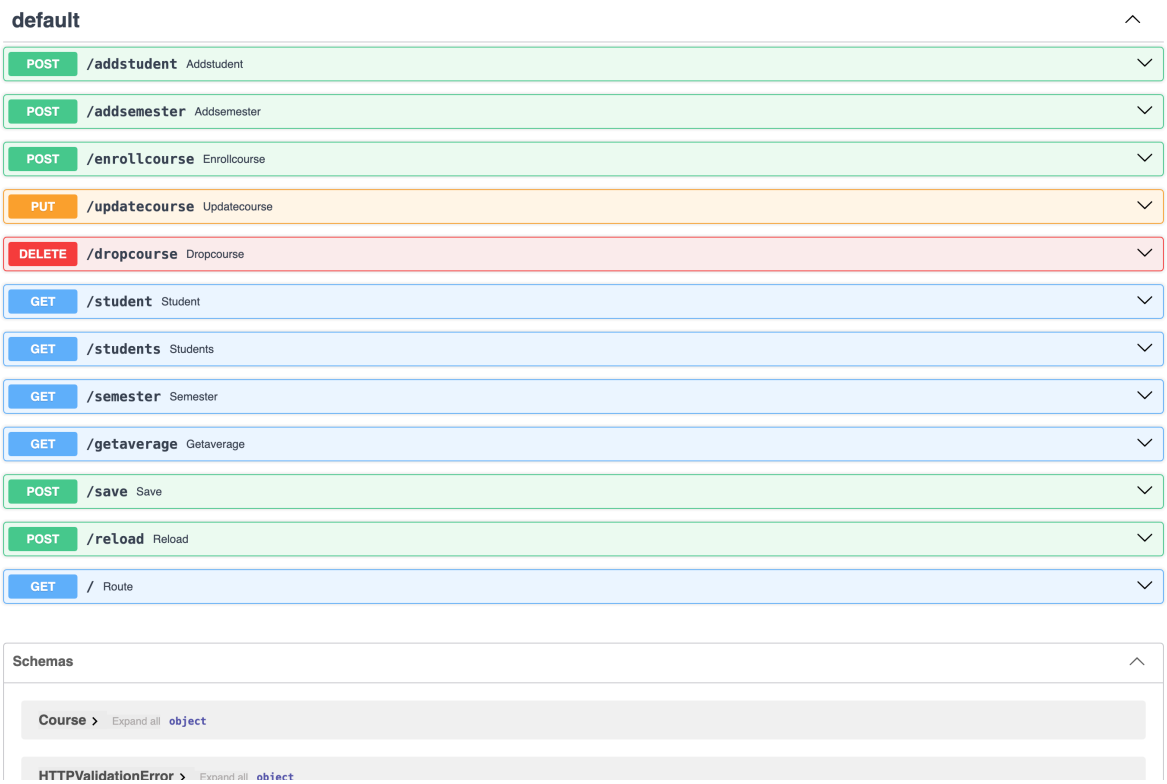
Egy félév rekordja a data.json fájlban (a tárgyak listájával együtt):

```
{
  "id": 101,
  "student_id": 1,
  "term": "2026/27/1",
  "courses": [
    {"course_id": 201, "code": "IP-18PROGEG", "name": "Programozás",
"credit": 5, "grade": 4},
    {"course_id": 203, "code": "IP-18PYTH", "name": "Python programozás",
"credit": 3, "grade": null}
  ]
}
```

A `grade` értéke `null`, amíg a tárgyhoz nincs beírva érdemjegy. Minden hallgatónak legfeljebb egy (az aktuális) féléve lehet.

Feladatok

Az itt látható végpontokat kell elkészítenünk:



- **Az addstudent segítségével egy hallgatót tudunk felvenni a rendszerbe. (5 pont)**
- **Az addsemester egy félévet rendel egy hallgatóhoz. (5 pont)**
- **Az enrollcourse egy tárgyat vesz fel az adott hallgató félévébe. (5 pont)**
- **Az updatecourse módosítja egy felvett tárgy adatait, például beírja az érdemjegyet. (10 pont)**
- **A dropcourse lead (töröl) egy tárgyat a félévből. (5 pont)**
- **A student egy adott hallgató adatait adja vissza. (5 pont)**
- **A students visszaadja az összes hallgatót. (5 pont)**
- **A semester egy hallgató félévében felvett összes tárgy adatait adja vissza. (5 pont)**
- **A getaverage egy hallgató kreditstatisztikáját adja vissza (összes és teljesített kredit, kreditsúlyozott átlag). (10 pont)**
- **A save és a reload az adatok mentésére és visszatöltésére szolgál; a visszatöltés a meglévő adatok cseréjével vagy összefésülésével is történhet. (5 + 10 pont)**
- A Route (/) a rendszer nevét adja vissza. Ez a funkció már implementálva van a main.py fájlban.

Az endpoint-ok teszteléséhez indítsuk el a REST API-t, és nézzük meg a listát a FastAPI által automatikusan generált webes felületen. Az API elindításakor a rendszer kiírja azt az URL-t és portot, amelyen a program fut. Ezt az URL-t a /docs végződéssel kiegészítve érjük el a webes felületet. Az alkalmazás elindítása:

```
uvicorn mainfileneve:appneve --reload
```

Az alkalmazás webes felületén (/docs) a fenti feladatok mind elvégezhetők. Minden funkció a routes.py fájlban és ebben a leírásban meghatározott módon viselkedik, és az ott leírt

adatokat adja vissza szabványos JSON formátumban, a JSONResponse függvénnyel, státuszkóddal együtt.

A végpontok megírása során az alábbi szabályokat kell betartani (10 pont):

- Minden végpontnál adjuk meg a response_model értékét (típus).
- Ügyeljünk a típusok megadására a függvények paramétereinél és visszatérési értékénél is.
- Ahol lehet, a függvények visszatérési értéke JSONResponse() legyen.
- Minden függvény tartalmazzon hibakezelést: hiba esetén dobjon HTTPException-t a megfelelő státuszkóddal.
- A függvények a JSON adatok mentéséhez és visszaolvasásához a filehandler.py és a filereader.py fájlt használják. Tegyük elérhetővé ezeket az alkalmazásban! Az adatokat a data.json fájlba kell menteni.
- A HTTP válaszok minden esetben tartalmazzák a megfelelő státuszkódot (lásd a következő fejezet táblázatát), és a végpontok a megfelelő HTTP metódust használják.

Az alkalmazás használata

A main.py fájl a REST API main modulja, ez indítja el az alkalmazást. A futtatásához telepíteni kell a pip csomagkezelővel a uvicorn, a fastapi és az e-mail cím validálásához szükséges pydantic[email] csomagokat:

```
pip install uvicorn fastapi "pydantic[email]"
```

vagy

```
pip3 install uvicorn fastapi "pydantic[email]"
```

Ezután az alkalmazás az API könyvtárában állva a következő paranccsal futtatható a terminálban:

```
uvicorn main:app --reload --port 9000
```

A port nem kötelező opció. A futó alkalmazás a következő URL-en érhető el:

```
http://127.0.0.1:9000
```

A webes UI (Swagger) pedig az alábbi URL használatával:

```
http://127.0.0.1:9000/docs
```

A dokumentáció pedig itt:

```
http://127.0.0.1:9000/redoc
```

A HTTP protokoll helyes használata

A REST API-ban a HTTP metódus fejezi ki, hogy mit csinál a végpont, a státuszkód pedig azt, hogy mi lett a kérés eredménye. A kérések és válaszok törzse mindig JSON formátumú (Content-Type: application/json). A végpontok megírásakor az alábbi táblázatokat kell követni.

HTTP metódus	**Mire használjuk**	**Példa ebben a feladatban**
GET	Adat lekérdezése, nem módosít semmit	/student, /students, /semester, /getaverage
POST	Új adat létrehozása	/addstudent, /addsemester, /enrollcourse
PUT	Meglévő adat módosítása (teljes csere)	/updatecourse
DELETE	Adat törlése	/dropcourse

Státuszkód	**Jelentése**	**Mikor használjuk**
200 OK	Sikeres kérés	Sikeres lekérdezés, módosítás, törlés, visszatöltés
201 Created	Létrejött az új erőforrás	Sikeres létrehozás (POST), mentés
400 Bad Request	Hibás kérés (üzleti szabály sérül)	pl. azonosító eltérés, limit túllépése, hibás fájlnév
404 Not Found	Nem található	Nem létező azonosító, fájl
409 Conflict	Ütközés a meglévő adatokkal	Már létező azonosító, már létező rekord, ellentmondásos adatok visszatöltéskor
422 Unprocessable Entity	A bemenő adat nem felel meg a sémának	Pydantic validációs hiba (ezt a FastAPI automatikusan adja)

Hibás kérés esetén a végpont **HTTPException**-t dobjon a megfelelő státuszkóddal és egy rövid, magyar nyelvű hibaüzenettel, pl.:

```
raise HTTPException(status_code=404, detail="Nem található hallgató a megadott ID-val: 99")
```

Opcionális: futtatás HTTPS protokollon (extra pont)

A uvicorn képes titkosított (HTTPS) kapcsolaton is kiszolgálni az API-t, ehhez egy tanúsítvány és egy hozzá tartozó kulcs kell. Fejlesztéshez elegendő egy saját aláírású (self-signed) tanúsítvány, amelyet az OpenSSL programmal lehet elkészíteni (macOS-en és Linuxon alapból telepítve van, Windowson a Git for Windows tartalmazza):

```
openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem -
days 365 -subj "/CN=localhost"
uvicorn main:app --reload --port 9443 --ssl-keyfile key.pem --ssl-certfile
cert.pem
```

Ezután az API a https://127.0.0.1:9443/docs címen érhető el. A böngésző figyelmeztetni fog, hogy a tanúsítvány nem megbízható. Ez saját aláírású tanúsítványnál normális, a figyelmeztetést el lehet fogadni. A Postmanben ilyenkor a Settings → SSL certificate verification beállítást kell kikapcsolni. A key.pem fájlt nem szabad beadni vagy megosztani!

Az adatok validálása (15 pont)

Az adatok kezelésére használható osztályokat a megadott séma alapján ki kell dolgozni. A schema.py tartalmazza az adatok küldésére és fogadására készített osztályokat. Az osztályokban az adatok legyenek validálva:

- Az int típusú azonosítók (id, student_id, course_id) nem lehetnek negatívak.
- Az email mező csak e-mail formátumot fogadhat el (EmailStr).
- A Neptun-kód pontosan 6 karakter, és csak nagybetűt (A–Z) és számjegyet (0–9) tartalmazhat. Kisbetűvel megadott kódot nagybetűssé alakítva kell eltárolni.
- A félév (term) formátuma ÉÉÉÉ/ÉÉ/F, pl. 2026/27/1, ahol F csak 1 vagy 2 lehet.
- A tárgy kódja 3–20 karakter, a neve nem lehet üres, a kreditértéke 1 és 30 közötti egész szám.
- Az érdemjegy 1 és 5 közötti egész szám, vagy null (még nincs beírva).
- Modell szintű validáció a Semester osztályban: a félév tárgyainak összkreditje legfeljebb 30 lehet, és egy course_id csak egyszer szerepelhet.
- Modell szintű validáció a DataStore osztályban (a teljes data.json): egyedi hallgatói azonosítók és Neptun-kódok, egyedi félév-azonosítók, egy hallgatónak legfeljebb egy féléve lehet, és minden félév létező hallgatóhoz tartozik. Ezt a mentés és a visszatöltés is használja.
- Hiba esetén ValueErrort kell dobni, és lehetőség szerint a kliens oldalon is jelezni kell a hibát. A FastAPI a modellből eredő hibát automatikusan 422-es válasszal jelzi.

Segítség a validáláshoz

Mezőszintű validáció (`field_validator`): a dekorált osztálymetódus megkapja a mező értékét. Hibás érték esetén ValueErrort dob, egyébként visszaadja az (esetleg átalakított) értéket. Szöveges minták ellenőrzésére a Python beépített `re` (reguláris kifejezés) modulja használható: a `re.fullmatch(minta, szöveg)` akkor ad vissza nem-None értéket, ha a teljes szöveg illeszkedik a mintára.

```
import re
from pydantic import BaseModel, field_validator

class Szoba(BaseModel):
    kod: str                        # pl. "A-101"

    @field_validator("kod")
    @classmethod
    def validate_kod(cls, value: str) -> str:
        value = value.strip().upper()          # átalakítás is lehetséges
        if not re.fullmatch(r"[A-Z]-\d{3}", value):  # [A-Z]: egy nagybetű,
\d{3}: 3 számjegy
            raise ValueError("A kód formátuma: betű-három számjegy, pl. A-
101")
        return value
```

Hasznos mintaelemek: `[A-Z0-9]` egy nagybetű vagy számjegy, `{6}` pontosan 6 ismétlés, `\d` számjegy, `[12]` az 1 vagy a 2 karakter, a `/` önmagát jelenti.

Opcionális mező: ha egy mező értéke hiányozhat, a típusa `Optional[int]` legyen, alapértéke pedig `None`, pl. `grade: Optional[int] = None`. A validátorban ilyenkor a None értéket is kezelni kell.

Modell szintű validáció (`model_validator`): akkor kell, ha a szabály több mezőt (vagy egy lista összes elemét) érinti. Az `after` módú validátor a mezők egyenkénti ellenőrzése után fut, és a már elkészült objektumot (self) kapja meg:

```
from pydantic import model_validator

class Rendeles(BaseModel):
    tetelek: List[Tetel]

    @model_validator(mode="after")
    def validate_tetelek(self) -> "Rendeles":
        if len(self.tetelek) > 10:
            raise ValueError("Legfeljebb 10 tétel lehet")
        return self
```

Validálás a végpontban: ha egy már tárolt (dict) adatot szeretnénk módosítás előtt ellenőrizni, a modell `model_validate()` metódusa használható. Hiba esetén `pydantic.ValidationError` kivétel keletkezik, amelyet el kell kapni, és HTTPException-né kell alakítani. Ez például az enrollcourse végpontban hasznos a 30 kredites korlát ellenőrzéséhez:

```
from pydantic import ValidationError

try:
    Semester.model_validate({**semester, "courses": semester["courses"] +
[course.model_dump()]})
except ValidationError as e:
    raise HTTPException(status_code=400, detail=e.errors()[0]["msg"])
```

A végpontok implementációja

Az alábbi végpontokat kell kidolgozni. A paraméterek neve és típusa a megadott legyen. Az egyszerű (nem modell típusú) paraméterek query paraméterek (pl. `/student?studentid=1`), a modell típusú paraméterek a kérés JSON törzsében érkeznek.

Hallgató felvétele a rendszerbe. A bemenet egy Student típus, a visszatérési érték a felvitt hallgató rekordja (201). Ha az azonosító vagy a Neptun-kód már foglalt: 409.

```
@routers.post('/addstudent', response_model=Student, status_code=201)
def addstudent(student: Student) -> Student:
    pass
```

Félév hozzáadása egy hallgatóhoz. A bemenő paraméterek a hallgató azonosítója és a félév megnevezése. Az új félév azonosítója a legnagyobb meglévő félév-azonosító + 1, a tárgylistája kezdetben üres. A visszatérési érték a következő szöveg: „Sikeres félév létrehozás.” (201). Nem létező hallgató: 404, ha a hallgatónak már van féléve: 409, hibás félév-formátum: 422.

```
@routers.post('/addsemester', response_model=str, status_code=201)
def addsemester(studentid: int, term: str) -> str:
    pass
```

Tárgy felvétele a hallgató félévébe. A bemenő paraméterek a hallgató azonosítója és a tárgy. A visszatérési érték a teljes félév a tárgyakkal (201). Ha a hallgatónak nincs féléve: 404, ha a tárgy már fel van véve: 409, ha a tárggyal a félév túllépné a 30 kreditet: 400.

```
@routers.post('/enrollcourse', response_model=Semester, status_code=201)
def enrollcourse(studentid: int, course: Course) -> Semester:
    pass
```

Példa kérés: `POST /enrollcourse?studentid=3`, a kérés törzse:

```
{"course_id": 301, "code": "IP-18SZAMEA", "name": "Számítógépes
architektúrák", "credit": 4, "grade": null}
```

Egy felvett tárgy adatainak módosítása (például jegy beírása). A bemenő paraméterek a hallgató azonosítója, a tárgy azonosítója, valamint a tárgy új adatai. A tárgy cseréjével is megoldható a feladat, de a tárgy azonosítója nem változhat: ha a törzsben lévő course_id eltér a courseid paramétertől, 400 a válasz. A módosítás után sem lépheti túl a félév a 30 kreditet (400). Nincs félév, vagy a tárgy nincs felvéve: 404. A visszatérési érték a teljes félév (200).

```
@routers.put('/updatecourse', response_model=Semester)
def updatecourse(studentid: int, courseid: int, course: Course) -> Semester:
    pass
```

Egy tárgy leadása (törlése) a hallgató félévéből. A bemenő paraméterek a hallgató azonosítója és a tárgy azonosítója. A visszatérési érték a teljes félév (200). Nincs félév vagy

nincs ilyen tárgy: 404.

```
@routers.delete('/dropcourse', response_model=Semester)
def dropcourse(studentid: int, courseid: int) -> Semester:
    pass
```

Egy adott hallgató adatainak megjelenítése. A bemenő paraméter a hallgató azonosítója, a visszatérési érték a hallgató rekordja (200). Nem létező hallgató: 404.

```
@routers.get('/student', response_model=Student)
def student(studentid: int) -> Student:
    pass
```

Az összes hallgató lekérdezése. Nincs bemenő paraméter, a félévek nem kerülnek megjelenítésre. A visszatérési érték a hallgatók listája (200).

```
@routers.get('/students', response_model=list[Student])
def students() -> list[Student]:
    pass
```

Egy adott hallgató félévében felvett tárgyak megjelenítése. A paramétere a hallgató azonosítója, a visszatérési érték a tárgyak listája (200). Üres félév esetén üres lista, ha nincs félév: 404.

```
@routers.get('/semester', response_model=list[Course])
def semester(studentid: int) -> list[Course]:
    pass
```

Egy adott hallgató kreditstatisztikája. A bemenő paraméter a hallgató azonosítója, a visszatérési érték egy Statistics objektum (200). Ha nincs félév: 404. A számítás szabályai:

- `total_credits`: az összes felvett tárgy kreditjének összege,
- `completed_credits`: a legalább 2-es érdemjeggyel rendelkező tárgyak kreditjének összege,
- `weighted_average`: a jeggyel rendelkező tárgyak (az 1-es is!) kreditsúlyozott átlaga, két tizedesjegyre kerekítve: Σ(kredit × jegy) / Σ(kredit). Ha egyetlen tárgynak sincs jegye, az értéke 0.0.

```
@routers.get('/getaverage', response_model=Statistics)
def getaverage(studentid: int) -> Statistics:
    pass
```

Példa: a mellékelt data.json alapján a `GET /getaverage?studentid=1` válasza:

```
{"student_id": 1, "term": "2026/27/1", "total_credits": 12,
"completed_credits": 9, "weighted_average": 3.56}
```

Mentés (5 pont): az API rendelkezik egy mentést segítő végponttal, ahol a mentésre szánt fájl neve (source) és a mentést tartalmazó célfájl (dest) a paraméterek. A

mentési és visszatöltési funkciókhoz tartozó fájlok a JSON adatokat kezelő fájlok könyvtárában (data) legyenek elhelyezve. A mentés a Schema alapján valósuljon meg: a beolvasott adatokat a DataStore modellel ellenőrizzük, és csak érvényes adatot írunk ki. Így a lehető legkevesebb adatvesztés vagy adathiba fordulhat elő.

```
@routers.post('/save', status_code=201)
def save(source: str = "data.json", dest: str = "backup.json") -> dict:
    pass
```

Visszatöltés (10 pont): az API rendelkezik egy mentett adatok visszatöltését segítő végponttal. A paraméterei: a visszatöltendő fájl neve (source), a célfájl (dest), a visszatöltés módja (mode), valamint az, hogy összefésüléskor megmaradjanak-e a csak a célfájlban lévő adatok (keep_extra). Sémának nem megfelelő fájl visszatöltése nem ronthatja el a data.json tartalmát!

```
@routers.post('/reload', response_model=ReloadResult)
def reload(source: str = "backup.json", dest: str = "data.json",
           mode: Literal["replace", "merge"] = "replace", keep_extra: bool =
True) -> ReloadResult:
    pass
```

A visszatöltés módjai:

mode	keep_extra	Mi történik
replace (alapértelmezett)	–	A célfájl (data.json) teljes tartalmát lecseréli a mentés tartalmára. A célfájlban lévő, a mentésben nem szereplő adatok elvesznek.
merge	true (alapértelmezett)	A mentésből csak azok az adatok kerülnek vissza, amelyek a célfájlban nincsenek meg. A csak a célfájlban lévő adatok megmaradnak.
merge	false	Mint az előző, de a csak a célfájlban lévő (a mentésben nem szereplő) adatok törlődnek.

Összefésüléskor (merge) az adatokat az azonosítójuk alapján kell párosítani: a hallgatót az id, a félévet a hallgató (student_id), a félévben felvett tárgyat a course_id alapján. Ha egy rekord mindkét fájlban megvan, a célfájl (data.json) változata marad meg, vagyis a visszatöltés a meglévő adatot nem írja felül. Az összefésülés a félévek tárgylistájára is vonatkozik: egy mindkét fájlban meglévő félévbe a mentésből a hiányzó tárgyak kerülnek be, keep_extra=false esetén pedig a csak a célfájlban lévő tárgyak törlődnek.

Az összefésült adatállománynak is ellentmondásmentesnek kell lennie (ezt a DataStore modell szintű validációja ellenőrzi). Ha például a mentésből visszatöltendő hallgató Neptun-kódja megegyezik egy csak a data.json-ban lévő hallgatóéval, a válasz 409 Conflict, és a data.json nem változik.

A válasz a visszatöltés eredményét is tartalmazza (ReloadResult): hány rekord került be (added), törlődött (removed), illetve változott meg (updated) a visszatöltés előtti állapothoz képest. A félévnél csak az id és a term számít (a tárgylista nem), a tárgyat a (student_id,

course_id) pár azonosítja. Példa (mode=merge, keep_extra=true, a mentésben egy új hallgató, annak féléve és egy új tárgy van):

```
{"message": "Sikeres visszatöltés.", "source": "backup.json", "dest":
"data.json",
 "mode": "merge", "keep_extra": true,
 "added":   {"students": 1, "semesters": 1, "courses": 1},
 "removed": {"students": 0, "semesters": 0, "courses": 0},
 "updated": {"students": 0, "semesters": 0, "courses": 0}}
```

Segítség: a mode paraméter értékkészlete a Python beépített typing moduljának `Literal` típusával adható meg. A FastAPI ilyenkor minden más értékre automatikusan 422-es hibát ad, a /docs felületen pedig legördülő listában kínálja fel a lehetséges értékeket. A bool típusú query paramétert a FastAPI a true/false (1/0, yes/no) értékekből alakítja át:

```
from typing import Literal

@routers.get('/pelda')
def pelda(sorrend: Literal["novekvo", "csokkeno"] = "novekvo", reszletes:
bool = False):
    ...
```

Az összefésüléshez és a változások összeszámolásához a filehandler.py `merge_data()` és `diff_counts()` függvényét kell megírni, a pontos szabályok a fájl leírásában találhatók. A rekordok kereséséhez érdemes azonosító szerinti szótárat (dict) vagy halmazt (set) készíteni, pl. `{s["id"]: s for s in data["Students"]}`; a bemenő adatok módosítása helyett a `copy.deepcopy()` függvénnyel másolatot kell készíteni.

Elvárt viselkedés mindkét végpontnál:

- Sikeres mentés esetén 201, sikeres visszatöltés esetén 200 a státuszkód. A mentés válasza: `{"message": "Sikeres mentés.", "source": "data.json", "dest": "backup.json"}`.
- Csak a data könyvtárban lévő, .json kiterjesztésű fájlnév adható meg. Könyvtárat tartalmazó név (pl. `../main.json`) vagy más kiterjesztés esetén 400 a válasz. Erre a filehandler.py `get_file_path()` függvénye szolgál.
- Nem létező forrásfájl esetén 404, sémának nem megfelelő vagy ellentmondásos tartalom esetén 422, ha a forrás és a cél azonos, akkor 400 a válasz. Visszatöltéskor a nem létező célfájl üres adatállománynak számít; ha az összefésült adatok ellentmondásosak, 409 a válasz, és a célfájl nem változik.

Segítség: a DataStore modell mezői a data.json kulcsai (Students és Semesters), így a teljes fájl egy lépésben ellenőrizhető:

```
data = load_json_file(source)                       # dict a fájlból
validated = DataStore.model_validate(data)     # ValidationError, ha hibás
save_json_file(validated.model_dump(mode="json"), dest)
```

Tesztelés (5 pont): a végpontokat a beépített tesztelési lehetőségek (/docs) mellett egy külső szoftverrel is teszteljük. Erre alkalmas lehet a Postman alkalmazás, de más szoftver (pl. Insomnia, Thunder Client, curl) is elfogadható. A tesztelésről rövid dokumentációt kell készíteni, amelyet a védés során az alkalmazással együtt be kell mutatni. A dokumentáció minden végpontnál tartalmazzon legalább egy sikeres és egy hibás kérést (a kérés, a várt és a kapott státuszkód, a válasz törzse).

Az endpoint-ok indokolt esetben módosíthatók, vagy ki is lehet őket egészíteni, de a végpontok neve, HTTP metódusa és paraméterei maradjanak a leírás szerintiek, mert a beadott megoldásokat automatikus teszttel is ellenőrizzük!

Kérdés esetén a gyakorlatvezetőhöz vagy az előadás vezetőjéhez lehet fordulni.

A feladat értékeléséhez minden végpontnak működnie kell. A pontszám a megvalósítás minőségétől is függ, vagyis ha egy végpont működik, de a megadott feltételeknek nem felel meg, akkor kevesebb pontot ér. A tesztelés is követelménye a feladat megoldásának!

A pontszámok összesítése

Részfeladat	**Pont**
addstudent végpont	5
addsemester végpont	5
enrollcourse végpont	5
updatecourse végpont	10
dropcourse végpont	5
student végpont	5
students végpont	5
semester végpont	5
getaverage végpont	10
Végpont szabályok (response_model, típusok, JSONResponse, hibakezelés, HTTP metódusok és státuszkódok)	10
Az adatok validálása (schema.py)	15
save végpont	5
reload végpont (replace / merge)	10
Tesztelés és dokumentációja	5
Összesen	**100**

Az elérhető maximális pontszám: 100 pont

Jó munkát!